

PERFORMANCE COMPARISON OF RELATIONAL DATABASE MANAGEMENT SYSTEMS

Pang Jee Nee

102771146

Takeru

102784225

Jessabel

102766540

Contents

Introduction2

Literature Review.....2

 Overview.....2

 Key Concepts2

 1. ACID properties2

 2. Indexing strategies3

 3. Query optimization techniques3

 4. Concurrency control mechanisms3

 5. Storage engines4

Methodology4

 Database Selection4

 Benchmarking Metrics and Workload Scenarios5

 Experimental Setup5

Performance Comparison6

 Query Execution Time:6

 Indexing Impact.....6

 Query Optimization Techniques7

 MySQL Execution Plans:7

 MariaDB Execution Plans:8

 PostgreSQL Execution Plans:8

 Concurrency Control and Locking.....8

 Storage Engine Comparison9

Introduction

Relational Database Management Systems (RDBMS) serve as the backbone of modern data-driven applications, handling data storage, retrieval, and management. As organizations increasingly rely on data-intensive applications, understanding the performance characteristics of different RDBMS becomes crucial. In this research, we explore the nuances of various RDBMS and compare their performance across critical dimensions to recommend best practices for optimizing performance.

Literature Review

Overview

Relational databases are foundational in many applications, requiring careful performance evaluation. This review summarises studies comparing different RDBMS focusing on essential concepts like ACID properties, indexing, and query optimisation to understand the literature's strengths, limitations, and gaps.

Key Concepts

1. ACID properties

According to Vithlani(2024), ACID stands for Atomicity, Consistency, Isolation and Durability. ACID properties are an array of crucial properties which assure the integrity of data transactions.

Atomicity ensures that each command in a transaction is handled as a single unit and that they all either succeed or fail together. This is necessary because it allows the users to be certain of the database's condition in the event of an unintentional incident, such as a crash or power outage. If any portion of the transaction failed, it would have either been successfully completed or rolled back (What are ACID Properties in Database Management Systems? 2024)

Consistency ensures that changes which are done during a transaction are to be compatible with database constraints. This comprises of all guidelines, limitations, and triggers. If the data gets into an illegal state, the entire transaction fails.

With Isolation, all transactions are guaranteed to occur in an isolated environment. As transactions don't conflict with one another, this makes it possible to conduct multiple transactions at once.

Durability ensures that changes are preserved in the database after the transaction is over. This guarantees that, even in the event of system failures such as crashes or power outages, data within the system will remain intact.

These properties ensure data reliability but implementing them may reduce performance under high load due to validation checks and locking.

2. Indexing strategies

According to Alooba, indexing strategies cover various techniques for organizing and structuring management databases (Alooba c. 2024). This includes exploring different types of indexes, such as B-Tree indexing, hash indexing, bitmap indexing, clustered indexing, and covering indexing. Each technique offers unique ways to optimize data retrieval and improve database performance. B-Tree is ideal for both range and equality queries due to hierarchical data arrangement. Hash is effective for exact-match queries by mapping data to unique identifiers. Bitmap is efficient for low-cardinality data by creating bitmaps of possible values. Clustered physically organises data for faster retrieval in range queries. Finally, Covering reduces table access by storing all relevant columns directly in the index.

3. Query optimization techniques

A key component of database management systems (DBMS) is query optimization, which looks at several query execution methodologies to find the most effective way to execute a particular query. Reducing the amount of system resources required to respond to the query and speeding up the outcome of responses are the objectives (Dremio c. 2024).

Fundamentally, query optimisation is the comparison of various query plans and selecting the one with the lowest projected cost. Key features of Query Optimization include:

- Parsing: Converts SQL queries into a query tree.
- Transformation: Uses optimisation, normalisation, and simplification to improve the query tree.
- Cost estimation: Calculates how much each possible execution plan would cost.
- Plan Selection: The most cost-effective plan is chosen and implemented by the DBMS.

4. Concurrency control mechanisms

Concurrency control mechanism in a database makes sure that concurrent transactions don't conflict with one another. Additionally, it avoids frequent problems like non-repeatable reads, missing updates, and dirty reads. The three main functions are preventing deadlocks, guaranteeing transaction serializability, and preserving data integrity (Dremio c. 2024).

There are two main types of concurrency control protocols, namely Optimistic Concurrency Control(OCC) and Pessimistic Concurrency Control(PCC):

- OCC assumes minimal conflict and checks at the end allowing the same database item to be accessed or updated by many transactions without obtaining locks, suitable for high-read systems (Adetunji 2024).
- PCC prevents conflicts by locking resources until the transaction is complete, ensuring correct serialisation.

Both methods handle high transaction volumes efficiently and scale well.

5. Storage engines

According to the Sisense Team (2024), different storage engines serve specific needs in MySQL

- InnoDB offers transaction support and ACID compliance for high-reliability applications.
- MyISAM is suitable for read-heavy environments that don't require transactions.

Choosing the right engine is crucial for performance and scalability.

Commented [JP1]: Critical analysis only if needed.

Methodology

Database Selection

This study compares the performance of three commonly used relational database management systems (RDBMS) which are MySQL, PostgreSQL, and MariaDB.

- MySQL is chosen due to popularity in web applications and is known for its reliability and high transactional speed when dealing with diverse data workloads.
- PostgreSQL is selected due to its powerful features and efficiency in managing complicated queries and large scale datasets, making it the best choice for at company level applications.
- MariaDB, almost same as MySQL, is included to object if its specific optimizations can boost any performance compared to processor.

These databases were selected to provide a broad perspective on the performance and capabilities of both open-source and commercial RDBMS under various conditions.

Benchmarking Metrics and Workload Scenarios

To assess the performance of each database system, the following metrics will be analysed:

- Response Time measures the time taken to complete a query, reflecting the speed of data retrieval.
- Throughput evaluates the number of transactions processed per unit of time, indicating the efficiency of the database under load.
- Scalability assesses the database's ability to maintain performance levels as the size of the dataset and number of concurrent users increase.

Workload scenarios to be tested include:

- Read-intensive Workload simulates a scenario dominated by read operations, typical in data analysis and reporting applications.
- Write-intensive Workload focuses on write operations relevant to transactional systems such as online transaction processing (OLTP) systems.

Experimental Setup

Hardware Configurations:

Processor: AMD Ryzen 7 6800H with Radeon Graphics, 3.20 GHz

Installed RAM: 16.0 GB

Software configurations:

Operating System: Windows with Windows Subsystem for Linux (WSL)

Database Configurations

- MySQL (ver 8.0.36): Installed on WSL, using an online dataset 'employees.sql' from https://github.com/datacharmer/test_db.
- MariaDB (ver 11.3.2): Installed in a Docker container on WSL, using the employees_partitioned.sql dataset and set to 3307.
- PostgreSQL: Optimised and installed on WSL, using a custom Python script for database setup and migration of SQL server to PostgreSQL using pgloader.

Tool Used

- Sysbench: For overall database performance benchmarking.
- Pgloader; Data loading tool for PostgreSQL which can migrate the whole database when needed.
- Python Scripts: Custom scripts to execute specific queries and measure performance.

Performance Comparison

Query Execution Time:

To evaluate the query execution time, the following queries were executed on each database.

1st Query SELECT * FROM employees WHERE emp_no = 10001

2nd Query SELECT COUNT(*) FROM salaries WHERE salary > 50000

Query	MySQL Execution Times(ms)	MariaDB Execution Times(ms)	PostgreSQL Execution Times(ms)
SELECT * FROM employees WHERE emp_no = 10001	0.5070368449	0.4672209422	1.052776972
SELECT COUNT(*) FROM salaries WHERE salary > 50000	289.4486586	633.5331599	94.435215

Table 1: Comparison of Query Execution Time

As observed from the table of results, the execution time for the first query is the fastest for MariaDB. This might be because it has an optimized query execution plan for simple primary key lookups and/or its indexing strategy or caching mechanisms could contribute to faster response time.

However, for the 2nd query PostgreSQL takes a clear lead in speed. Its query planner might optimize aggregation queries more efficiently and utilizes its parallel processing in a less limited way.

Indexing Impact

To evaluate query performance with and without indexes, indexes were created to evaluate their impact:

“CREATE INDEX idx_emp_no ON employees(emp_no)”

“CREATE INDEX idx_salary ON salaries(salary)”

The queries evaluated were the same as those used in the Query Execution Time section:

Before Index

Database	Query1(ms)	Query2(ms)
MySQL	0.534	261.237
MariaDB	0.484	606.877
PostgreSQL	1.107	84.741

Table 2: Before Indexing Database

After Index

Database	Query1(ms)	Query2(ms)
MySQL	1.052	333.671
MariaDB	1.057	620.910
PostgreSQL	1.456	70.700

Table 3: After Indexing Database

The two queries used in testing the indexing impact actually represents two scenarios. Query1 performs an equality search which represents a common use case where the user searches for a specific record based on a unique identifier. Query2 involves an aggregation function (COUNT(*)) and a range condition (salary > 50000).

In the case of Query1, indexing didn't lead to improvements in performance in all the databases. As it's a simple query which is efficient even without indexing. Indexing primarily helps with range queries, joins, and more complex conditions. Since Query1 is straightforward, the impact of indexing is less pronounced.

However, in Query2, PostgreSQL is seen to be the only database with an improvement in efficiency with a huge difference of speed compared to the MySQL and MariaDB. It could be due to the fact that PostgreSQL has a sophisticated parallel execution framework. When executing complex queries, it can split the work across multiple worker processes, improving overall throughput.

Query Optimization Techniques

The query execution plans were analyzed by using the same queries from the query execution time experiment.

MySQL Execution Plans:

```
SELECT * FROM employees WHERE emp_no = 10001
```

Plan: [(1, 'SIMPLE', 'employees', None, 'const', 'PRIMARY,idx_emp_no', 'PRIMARY', '4', 'const', 1, 100.0, None)]

Analysis: Uses 'const' access type, leveraging the primary key and 'idx_emp_no' index. Efficient for single row retrieval.


```
SELECT COUNT(*) FROM salaries WHERE salary > 50000
```

Plan: [(1, 'SIMPLE', 'salaries', None, 'range', 'idx_salary', 'idx_salary', '4', None, 1419213, 100.0, 'Using where; Using index')]

Analysis: Uses 'range' access type with 'idx_salary' index for efficient filtering and counting.

MariaDB Execution Plans:

```
SELECT * FROM employees WHERE emp_no = 10001
```

Plan: [(1, 'SIMPLE', 'employees', 'const', 'PRIMARY,idx_emp_no', 'PRIMARY', '4', 'const', '1', '')]

Analysis: Like MySQL, MariaDB uses the 'const' access type, effectively using the primary key and 'idx_emp_no' index for efficient retrieval.

```
SELECT COUNT(*) FROM salaries WHERE salary > 50000
```

Plan: [(1, 'SIMPLE', 'salaries', 'range', 'idx_salary', 'idx_salary', '4', None, '2834326', 'Using where; Using index')]

Analysis: Uses 'range' access type with 'idx_salary' index, optimizing filtering and counting.

PostgreSQL Execution Plans:

```
SELECT * FROM employees WHERE emp_no = 10001
```

Plan: [(Index Scan using employees_pkey on employees (cost=0.42..8.44 rows=1 width=29)), (' Index Cond: (emp_no = 10001)')]

Analysis: PostgreSQL uses an 'Index Scan' on the primary key, efficiently retrieving a single row retrieval.

```
SELECT COUNT(*) FROM salaries WHERE salary > 50000
```

Plan: [(Finalize Aggregate (cost=33449.48..33449.49 rows=1 width=8)), (' -> Gather (cost=33449.26..33449.47 rows=2 width=8)), (' Workers Planned: 2'), (' -> Partial Aggregate (cost=32449.26..32449.27 rows=1 width=8)), (' -> Parallel Seq Scan on salaries (cost=0.00..30186.74 rows=905008 width=0)), (' Filter: (salary > 50000))]

Analysis: PostgreSQL uses a parallel sequential scan and aggregation to count rows. The use of parallelism helps to speed up the query, but the absence of an index scan indicates potential for further optimisation.

Concurrency Control and Locking

The graph below illustrates the response times for MySQL, MariaDB and PostgreSQL as the number of concurrent transactions increases from 1 to 10.

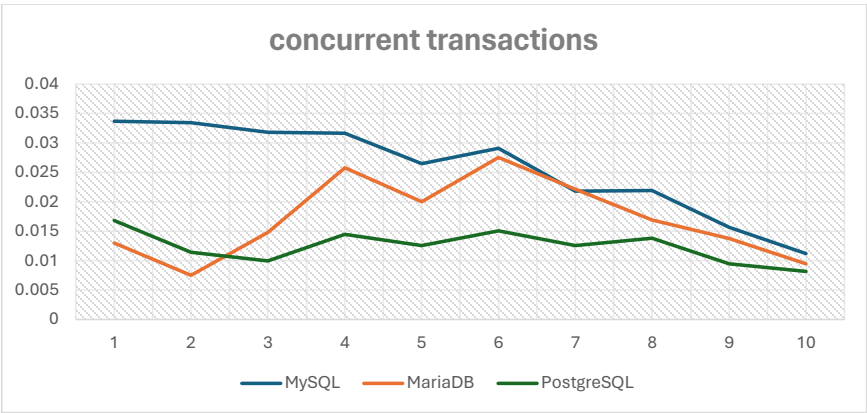


Table 4: Response Times vs Concurrent Transactions

MySQL starts with relatively low response times (around 0.04 seconds) for a single concurrent transaction. As the number of concurrent transactions increases, MySQL maintains relatively stable performance using traditional locking mechanism such as row-level locking. The response times fluctuate slightly but remain within a reasonable range. MySQL handles concurrency reasonably well, but there's no significant improvement or degradation as the load increases.

MariaDB begins with the lowest initial response times (around 0.013 seconds) for a single transaction. Interestingly, as the number of concurrent transactions increases, MariaDB's response times improve due to its adaptive row-level locking strategy ensures efficient access to data. It crosses over MySQL's performance at a point. MariaDB adapts better to increased concurrency, possibly due to its internal optimizations or parallel processing capabilities.

PostgreSQL consistently exhibits low response times across all levels of concurrency with the initial response time at around 0.02 seconds. Unlike MySQL and MariaDB, PostgreSQL doesn't show significant fluctuations. It maintains its efficiency even as the workload increases due to PostgreSQL's multiversion concurrency control (MVCC) system and parallel execution. PostgreSQL excels in handling concurrent transactions, demonstrating stability and predictability.

Storage Engine Comparison

Assessment of the impact of different storage engines (if applicable):

MySQL using InnoDB

- Insert Time: 1701.41 ms
- Features: Supports transactions and ACID compliance for high-reliability applications

MySQL MyISAM

- Insert Time: 15400.22 ms
- Features: Optimised for read-heavy environments, lacks transaction support.

MariaDB InnoDB

- Insert Time: 1890.01 ms
- Features: Similar to MySQL InnoDB, with transaction support and ACID compliance.

MariaDB MyISAM

- Insert Time: 1876.60 ms
- Features: Similar to MySQL MyISAM, optimised for reads, no transaction support.

Result and Discussion

Query Execution Time Results

From these results, we observe that MariaDB performs the fastest for the first query (single-row retrieval), indicating an optimized primary key lookup and efficient caching mechanisms. For the second query (aggregation function), PostgreSQL significantly outperforms MySQL and MariaDB, demonstrating its robust handling of complex queries and effective parallel processing capabilities.

Impact of Indexing

Query1 (single-row retrieval) result: Adding indexes did not significantly improve performance and even caused a slight delay. This is because single-row retrieval is already efficient, and the overhead of managing the index can negate any benefits.

Query2 (aggregation function) result: PostgreSQL showed a significant performance improvement with indexing, reducing execution time considerably. This highlights PostgreSQL's efficient use of indexes for complex queries.

Concurrency Control and Locking

MySQL maintains stable performance with increasing concurrency, utilizing row-level locking. MariaDB improves response times as concurrency increases, likely due to adaptive row-level locking strategies. PostgreSQL consistently exhibits low response times across all concurrency levels, benefiting from its MVCC system and parallel execution.

Storage Engine Comparison

Unexpected Results and Anomalies

Performance Decrease after Indexing in Query1: For single-row retrieval, adding an index can introduce overhead, leading to a slight performance decrease.

MariaDB's Performance in Aggregation Queries: MariaDB underperforms in complex aggregation queries, indicating potential inefficiencies in its handling of such queries.

Query Execution Time Results

From these results, we observe that MariaDB performs the fastest for the first query (single-row retrieval), indicating an optimized primary key lookup and efficient caching mechanisms. For the second query (aggregation function), PostgreSQL significantly outperforms MySQL and MariaDB, demonstrating its robust handling of complex queries and effective parallel processing capabilities.

Recommendations and Best Practices

Database Selection

For applications that are read-heavy, MariaDB with the MyISAM storage engine is optimal due to its fast read performance. When dealing with write-intensive applications, MySQL or MariaDB with the InnoDB storage engine is preferable because they support transactions and are ACID-compliant, ensuring high reliability. For handling complex queries, PostgreSQL is recommended because of its efficient query processing and advanced parallel processing capabilities.

Indexing Strategies

For equality searches, primary key indexing provides minimal performance benefits, so it is more effective to focus on optimizing more complex queries. However, for range queries and aggregations, using indexes can significantly improve performance. This is especially true for databases like PostgreSQL, which leverage indexing effectively to enhance query performance.

Query Tuning

To optimize query performance, it is essential to regularly analyze query execution plans. This practice helps in identifying opportunities for optimization and ensures that the database queries are running as efficiently as possible. Additionally, for complex workloads, leveraging parallel processing can greatly enhance performance. PostgreSQL, in particular, supports parallel query execution, making it a powerful tool for optimizing complex queries.

Conclusion

This research emphasizes the importance of understanding database performance characteristics for application development. Each database exhibits unique strengths and weaknesses, making them suitable for different scenarios. By selecting the appropriate database and employing effective optimization techniques, developers can significantly enhance application performance. Future Research Future research could explore the performance of NoSQL databases and hybrid models, providing a broader perspective on database performance optimization. Additionally, investigating the impact of different hardware configurations and real-world workloads could further refine our understanding of database performance.

References

Adetunji, H 2024, *How Databases Guarantee Isolation – Pessimistic vs Optimistic Concurrency Control Explained*, freeCodeCamp, viewed 21 May 2024, <<https://www.freecodecamp.org/news/how-databases-guarantee-isolation/#optimistic-concurrency-control>>.

Alooba c. 2024, *Indexing Strategies*, Alooba, viewed 20 May 2024, <<https://www.alooba.com/skills/concepts/database-and-storage-systems/database-management/indexing-strategies/#:~:text=In%20simple%20terms%2C%20indexing%20strategies,quickly%20locate%20specific%20data%20entries.>>.

Dremio c. 2024, *Concurrency Control*, Dremio, viewed 21 May 2024, <<https://www.dremio.com/wiki/concurrency-control/#:~:text=Concurrency%20Control%20ensures%20that%20simultaneous,transactions%2C%20and%20maintaining%20data%20integrity.>>.

Dremio c. 2024, *Query Optimization*, Dremio, viewed 20 May 2024, <<https://www.dremio.com/wiki/query-optimization/>>.

Sisense Team 2024, 'The Beginner's Guide to MySQL Storage Engines', *Sisense*, viewed 21 May 2024, <<https://www.sisense.com/blog/beginners-guide-to-mysql-storage-engines/>>.

Vithlani, H 2024, 'ACID Properties in DBMS', 30 April, viewed 19 May 2024, <<https://www.scaler.com/topics/dbms/acid-properties-in-dbms>>.

What are ACID Properties in Database Management Systems?, 2024, MongoDB, viewed 19 May 2024, < <https://www.mongodb.com/resources/basics/databases/acid-transactions#:~:text=of%20unexpected%20errors.-,Atomicity,the%20state%20of%20the%20database.>>.